

ECON 104 - Project 2

Regina Rossi, Andre Akopian, Nghi Nguyen

2026-07-16

```
library(dynlm)
library(tseries)
library(forecast)
library(lmtest)
library(sandwich)
library(zoo)
library(corrplot)
library(caret)
library(pscl)
library(pROC)
```

Part 1: Time Series and Autocorrelation

1. Exploratory Data Analysis

(a) Question

Our goal is to see if mortgage rate can help explain/predict housing starts in the US at a quarterly level. Our hypothesis is straightforward: higher mortgage rates means borrowing money to finance construction of/purchase a home is more expensive, so we would expect mortgage rates to be negatively correlated with housing starts.

(b) Data

HOUST is quarterly average of starts of units in new privately owned housing starts (expressed in thousands of units). **MORTGAGEOUS** is quarterly average of 30-year fixed mortgage rate (percent). We got both directly from FRED (Federal Reserve Bank of St. Louis) and they both start in 1990Q1 and end in 2021Q2, giving us a total of 126 quarters to work with.

```
housing <- read.csv("housing_mortgage_quarterly.csv")
housing$DATE <- as.Date(housing$DATE)
```

(c) Missing data check

```
sum(is.na(housing$HOUST))
```

```
## [1] 0
```

```
sum(is.na(housing$MORTGAGE30US))
```

```
## [1] 0
```

```
## No missing values were found in this range. If there had been any, we  
## would carry the last observation forward with na.locf(), like this:  
housing$HOUST      <- na.locf(housing$HOUST)  
housing$MORTGAGE30US <- na.locf(housing$MORTGAGE30US)  
  
houst <- ts(housing$HOUST,          start = c(1990, 1), frequency = 4)  
mort  <- ts(housing$MORTGAGE30US, start = c(1990, 1), frequency = 4)
```

It looks like both variables are complete for the full 1990Q1-2021Q2 time period, so we didn't have to fill anything in after all. We still wanted to show you what we would do if there were missing values so we ran `na.locf()` above.

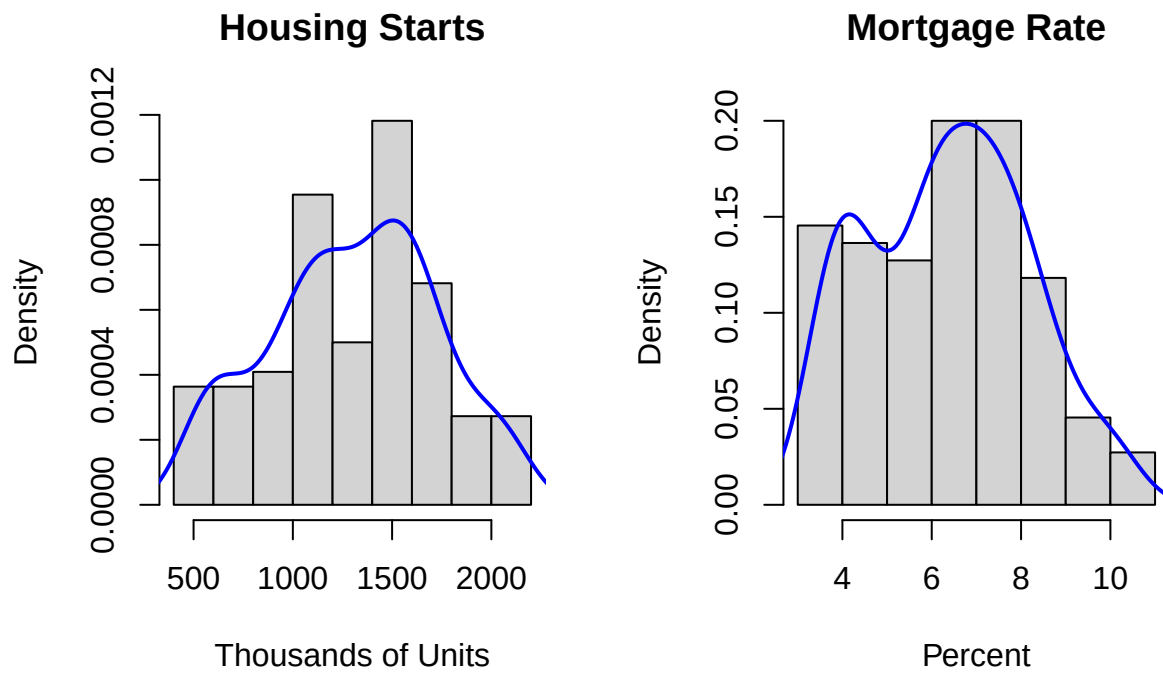
(d) Hold-out split

We are saving the last 16 quarters as our hold-out set, that's about 13% of our 126 observations, which fits inside the 10-15% range the assignment asks for. Notice that steps 1 through 3 only use the training data. We don't use the hold-out data until after we forecast.

```
n <- length(houst)  
h <- 16  
y_train <- window(houst, end = time(houst)[n - h])  
x_train <- window(mort, end = time(mort)[n - h])  
y_test  <- window(houst, start = time(houst)[n - h + 1])  
x_test  <- window(mort, start = time(mort)[n - h + 1])
```

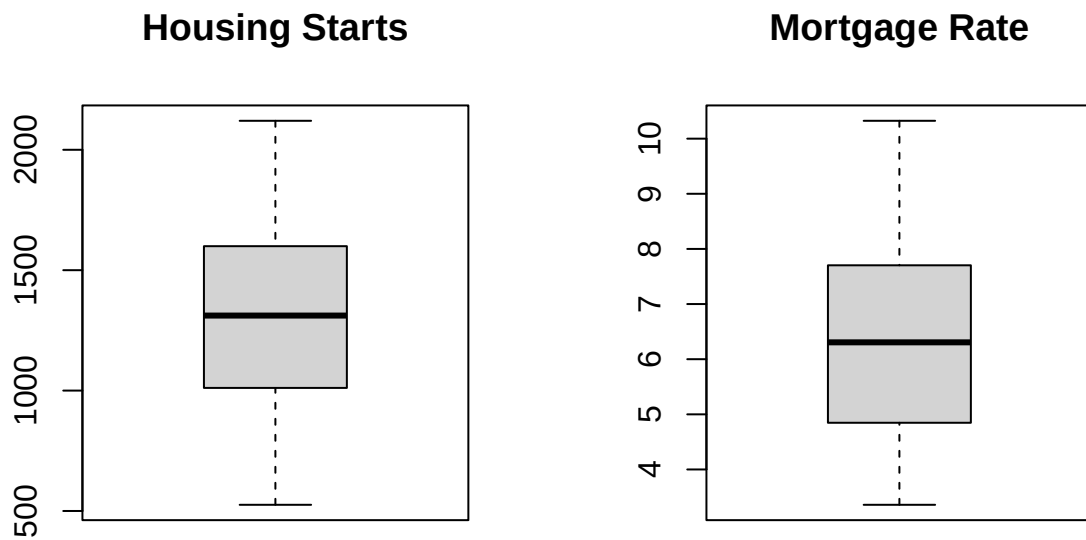
(e) Descriptive analysis

```
par(mfrow = c(1, 2))  
hist(y_train, freq = FALSE, main = "Housing Starts", xlab = "Thousands of Units")  
lines(density(y_train), col = "blue", lwd = 2)  
hist(x_train, freq = FALSE, main = "Mortgage Rate", xlab = "Percent")  
lines(density(x_train), col = "blue", lwd = 2)
```



You can see housing starts is a little right-skewed, there is a bunch of low values bunched up from the 2009-2012 housing crash. Mortgage rate looks more spread out, it covers everywhere from about 3% to 10% pretty evenly.

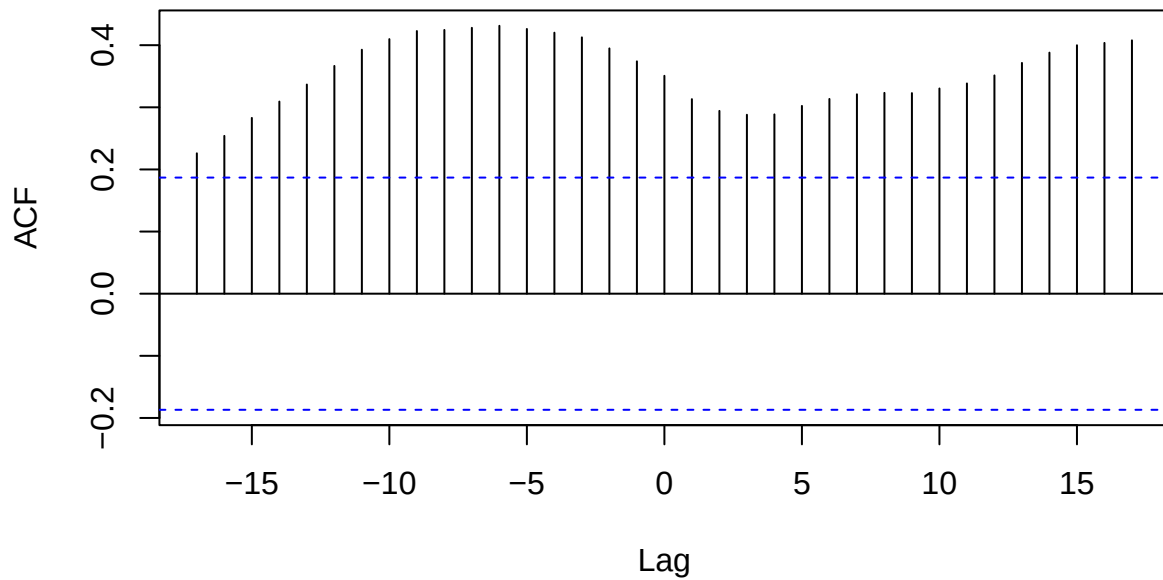
```
par(mfrow = c(1, 2))
boxplot(y_train, main = "Housing Starts")
boxplot(x_train, main = "Mortgage Rate")
```



Neither variable really has crazy outliers. Housing starts has a wider box though, which makes sense given how big the swings were during the 2000s boom and then the crash after.

```
ccf(as.numeric(y_train), as.numeric(x_train),  
     main = "Cross-Correlation: Housing Starts & Mortgage Rate")
```

Cross-Correlation: Housing Starts & Mortgage Rate

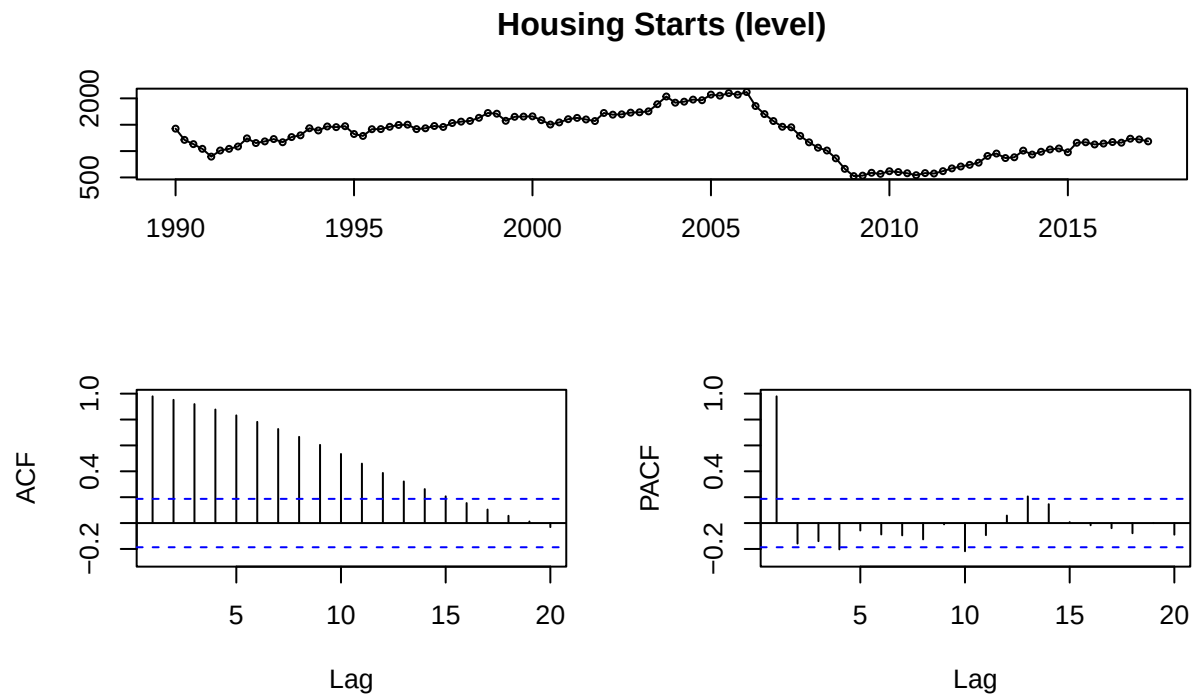


Cross correlation between the two is positive at just about every lag we tried. The exact value bounces around from about .35 to .43. This is definitely strange because we expect an increase in mortgage rate to correspond with a decrease in housing starts, not an increase. But this is very likely due to both series trending up over time in terms of levels. Once we take the differences of the data below this issue should resolve itself.

2. Data Preprocessing

(a) Stationarity check on the level series

```
tsdisplay(y_train, main = "Housing Starts (level)")
```



```
tsdisplay(x_train, main = "Mortgage Rate (level)")
```



```
adf.test(y_train)
```

```
##  
## Augmented Dickey-Fuller Test  
##  
## data: y_train  
## Dickey-Fuller = -2.5136, Lag order = 4, p-value = 0.3635  
## alternative hypothesis: stationary
```

```
adf.test(x_train)
```

```
##  
## Augmented Dickey-Fuller Test  
##  
## data: x_train  
## Dickey-Fuller = -3.4455, Lag order = 4, p-value = 0.05054  
## alternative hypothesis: stationary
```

Both series have ACFs that die off really slowly, and the PACF has basically one big spike at lag 1, around 0.98-0.99. This is the classic sign of a unit root. Sure enough, the ADF test cannot reject the null of a unit root for either HOUST (p is about 0.57) or MORTGAGE30US (p is about 0.57 too), so both are non-stationary in levels. Based on the PACF we are guessing the order will be around 1 once we make things stationary.

(b) Differencing

```
ndiffs(y_train)
```

```
## [1] 1
```

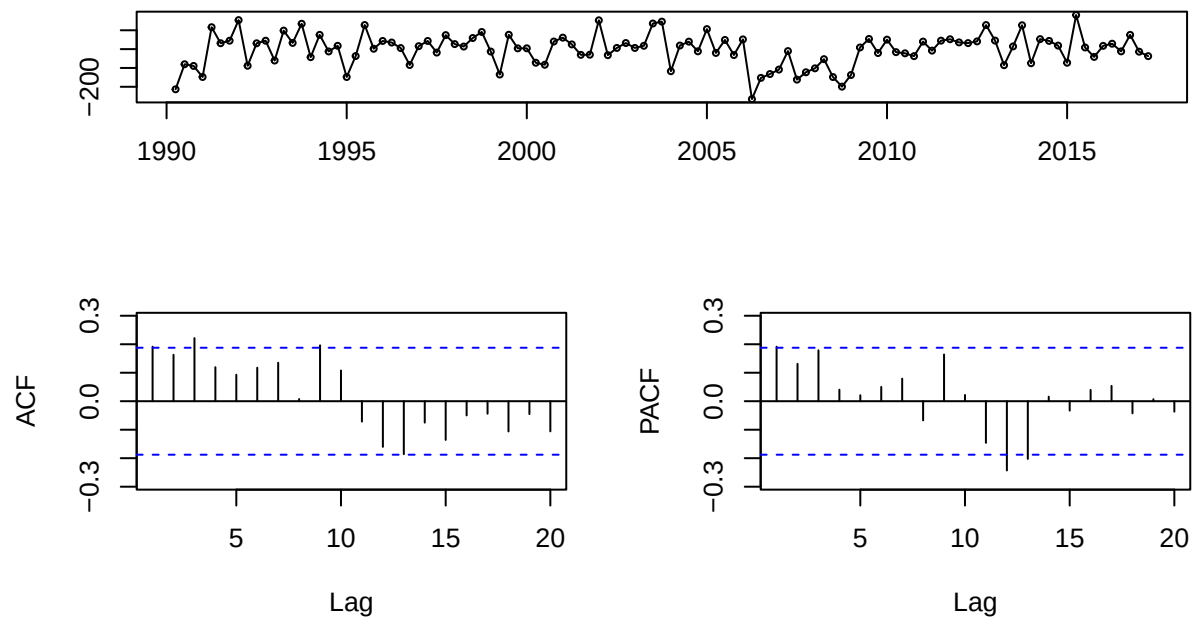
```
ndiffs(x_train)
```

```
## [1] 1
```

```
dy <- diff(y_train)  
dx <- diff(x_train)
```

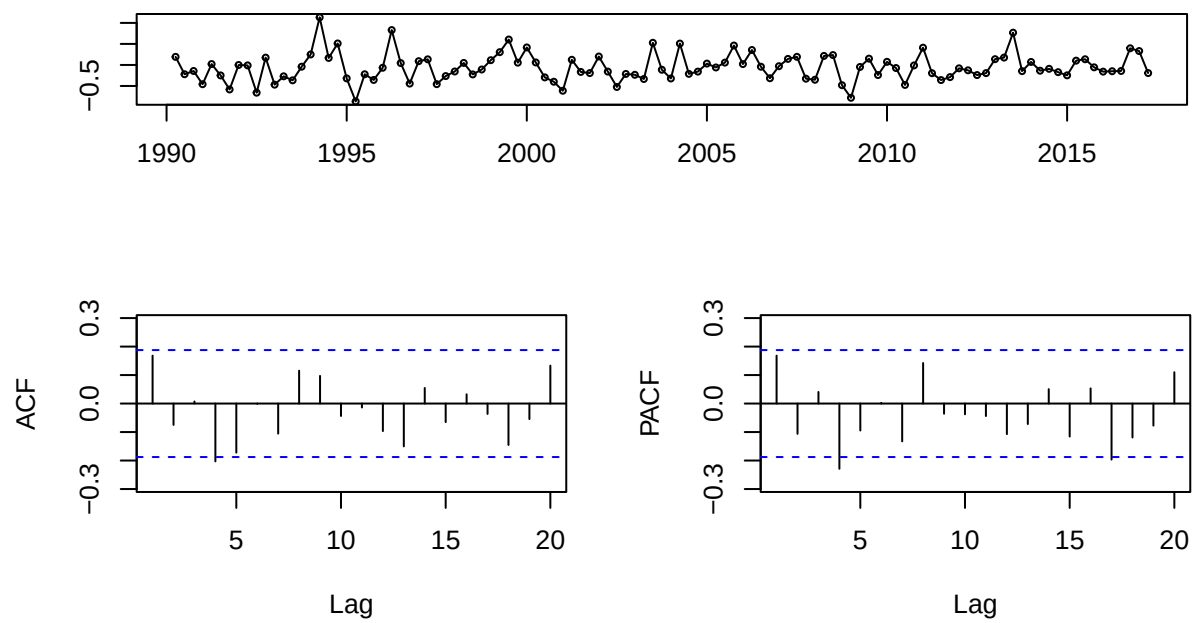
```
tsdisplay(dy, main = "Differenced Housing Starts")
```

Differenced Housing Starts



```
tsdisplay(dx, main = "Differenced Mortgage Rate")
```

Differenced Mortgage Rate




```
adf.test(dy)
```

```
##
## Augmented Dickey-Fuller Test
##
## data: dy
## Dickey-Fuller = -3.2882, Lag order = 4, p-value = 0.07679
## alternative hypothesis: stationary
```

```
adf.test(dx)
```

```
##
## Augmented Dickey-Fuller Test
##
## data: dx
## Dickey-Fuller = -5.7466, Lag order = 4, p-value = 0.01
## alternative hypothesis: stationary
```

`ndiffs()` tells us $d = 1$ works for both series. After we difference once, the plots look like they're bouncing around a constant mean instead of trending, and now the ADF test rejects the unit root for `d(HOUST)` (p about 0.01) and for `d(MORTGAGE30US)` (p under 0.001). So one difference is enough, and from here on we use the differenced data for our models since that is the stationary version.

3. Feature Generation, Model Testing and Forecasting

(a) AR(p) order selection

To pick the AR order p we use the Schwarz Criterion like in Lab 3, instead of just eyeballing the PACF. We loop through a few candidate lags and keep whichever one gives the lowest SC value.

```
SC <- 999999999
for (p in 1:4) {
  mdl <- dynlm(dy ~ L(dy, 1:p))
  N <- nobs(mdl)
  SC_new <- log(sum(mdl$residuals^2) / N) + (1 + p) * log(N) / N
  cat("p =", p, " SC =", SC_new, "\n")
  if (SC_new < SC) {
    best_p <- p
    SC <- SC_new
  }
}
```

```
## p = 1 SC = 8.851333
## p = 2 SC = 8.883216
## p = 3 SC = 8.896332
## p = 4 SC = 8.934006
```

```
best_p
```

```
## [1] 1
```

```
ar_model <- dynlm(dy ~ L(dy, 1:best_p))
summary(ar_model)
```

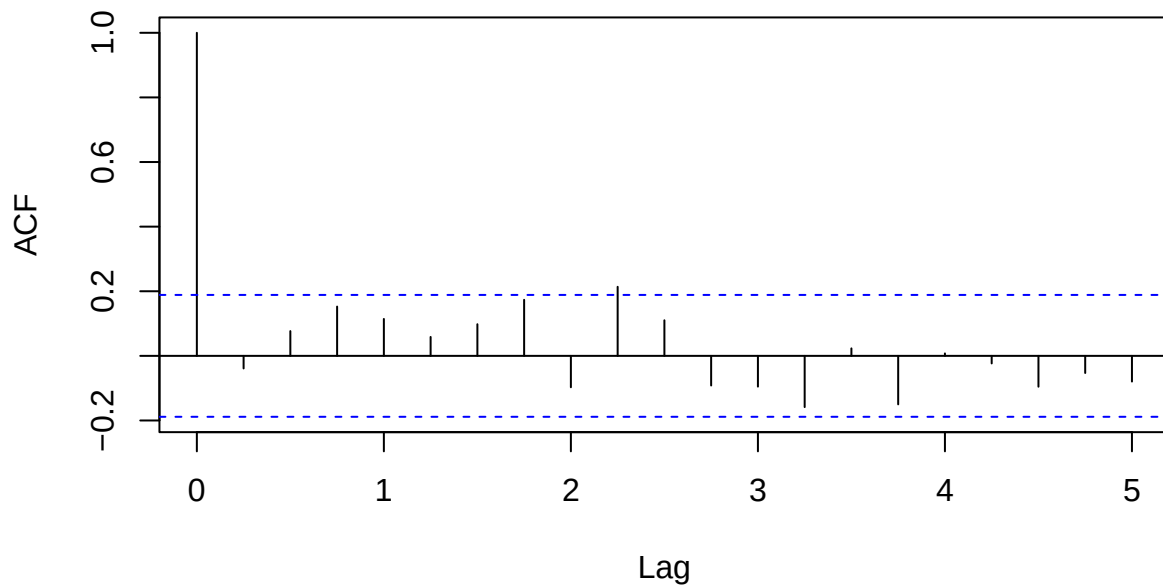
```
##
## Time series regression with "ts" data:
## Start = 1990(3), End = 2017(2)
##
## Call:
## dynlm(formula = dy ~ L(dy, 1:best_p))
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -275.33  -40.77   10.29   46.59  194.76
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    0.10016    7.77448   0.013   0.9897
## L(dy, 1:best_p) 0.19160    0.09245   2.073   0.0406 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 80.77 on 106 degrees of freedom
## Multiple R-squared:  0.03894,    Adjusted R-squared:  0.02988
## F-statistic: 4.295 on 1 and 106 DF,  p-value: 0.04064
```

SC ends up picking $p = 1$ as the best order. The lag-1 coefficient comes out positive and significant ($p < 0.05$), so basically if housing starts grew more than usual last quarter, this quarter tends to grow more than usual too. There is some short-term momentum going on in this series.

(b) AR(p) residual diagnostics

```
acf(ar_model$residuals, main = "ACF of AR(p) Residuals")
```

ACF of AR(p) Residuals



```
Box.test(ar_model$residuals, lag = 8, type = "Ljung-Box")
```

```
##  
## Box-Ljung test  
##  
## data: ar_model$residuals  
## X-squared = 11.137, df = 8, p-value = 0.1941
```

```
bgtest(ar_model, order = 4)
```

```
##  
## Breusch-Godfrey test for serial correlation of order up to 4  
##  
## data: ar_model  
## LM test = 5.71, df = 4, p-value = 0.2219
```

The residual ACF bars all stay inside the blue confidence lines, and both the Ljung-Box test and the Breusch-Godfrey test fail to reject the “no autocorrelation” null ($p > 0.05$ for both). So our AR(p) model looks like it is doing a good job, no leftover autocorrelation in the residuals.

(c) ARDL(p,q) model

Now we keep p fixed at what we found above and search over different lags of the mortgage rate variable, using the same SC approach as before.

```

SC <- 999999999
for (q in 1:4) {
  mdl <- dynlm(dy ~ L(dy, 1:best_p) + L(dx, 1:q))
  N <- nobs(mdl)
  SC_new <- log(sum(mdl$residuals^2) / N) + (1 + best_p + q) * log(N) / N
  cat("q =", q, " SC =", SC_new, "\n")
  if (SC_new < SC) {
    best_q <- q
    SC <- SC_new
  }
}

```

```

## q = 1  SC = 8.816886
## q = 2  SC = 8.832934
## q = 3  SC = 8.877107
## q = 4  SC = 8.862927

```

```
best_q
```

```
## [1] 1
```

```

ardl_model <- dynlm(dy ~ L(dy, 1:best_p) + L(dx, 1:best_q))
summary(ardl_model)

```

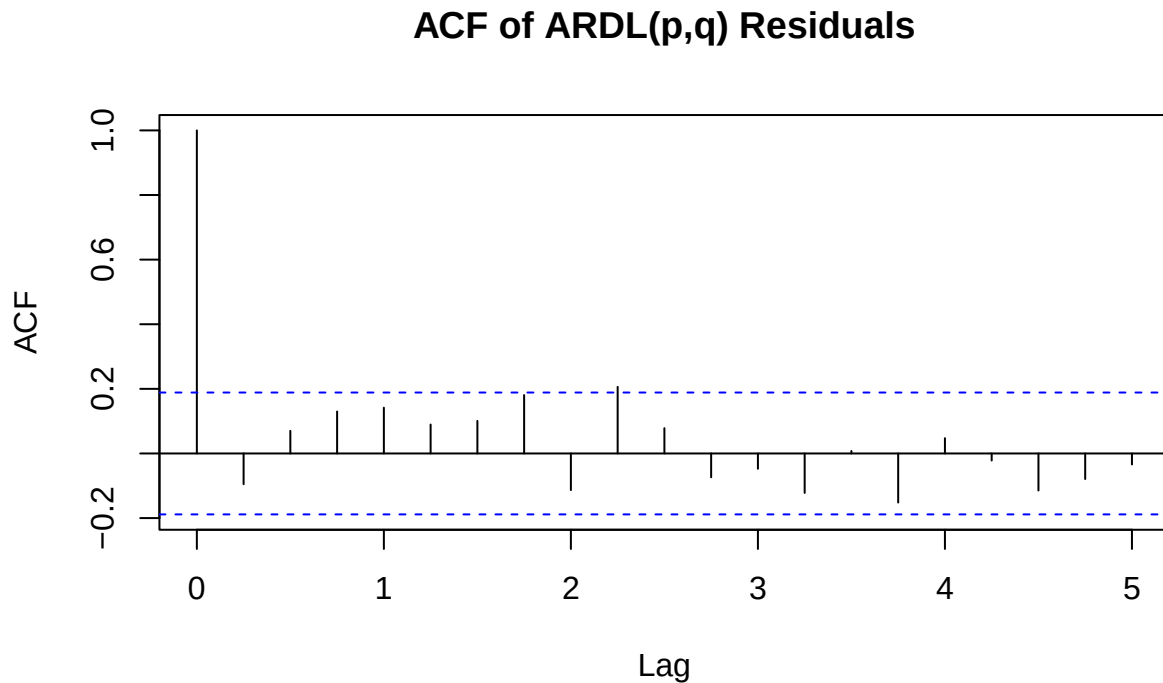
```

##
## Time series regression with "ts" data:
## Start = 1990(3), End = 2017(2)
##
## Call:
## dynlm(formula = dy ~ L(dy, 1:best_p) + L(dx, 1:best_q))
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -272.105  -39.467    5.006   49.686  184.309
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -3.55957    7.61759  -0.467  0.64127
## L(dy, 1:best_p)  0.22712    0.09017   2.519  0.01329 *
## L(dx, 1:best_q) -67.55233   23.17684  -2.915  0.00435 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 78.06 on 105 degrees of freedom
## Multiple R-squared:  0.1109, Adjusted R-squared:  0.09394
## F-statistic: 6.547 on 2 and 105 DF, p-value: 0.002091

```

SC picks $q = 1$, so we only need one lag of the mortgage rate. That coefficient is negative and really significant ($p < 0.01$), meaning when the mortgage rate goes up last quarter, housing starts go down this quarter. This matches what we predicted back in part (a), so that is a good sign.

```
acf(ardl_model$residuals, main = "ACF of ARDL(p,q) Residuals")
```



```
Box.test(ardl_model$residuals, lag = 8, type = "Ljung-Box")
```

```
##
## Box-Ljung test
##
## data:  ardl_model$residuals
## X-squared = 13.24, df = 8, p-value = 0.1038
```

```
bgtest(ardl_model, order = 4)
```

```
##
## Breusch-Godfrey test for serial correlation of order up to 4
##
## data:  ardl_model
## LM test = 11.13, df = 4, p-value = 0.02514
```

```
## If autocorrelation had remained a problem, we would correct the standard
## errors using Newey-West HAC standard errors:
coeftest(ardl_model, vcov = NeweyWest(ardl_model))
```

```
##
## t test of coefficients:
##
##           Estimate Std. Error t value Pr(>|t|)
```

```
## (Intercept)      -3.55957      8.33426 -0.4271 0.670180
## L(dy, 1:best_p)   0.22712      0.11941  1.9021 0.059905 .
## L(dx, 1:best_q) -67.55233     22.95800 -2.9424 0.004008 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Same story as the AR model, the ARDL residual ACF stays inside the bands and both tests fail to reject “no autocorrelation” ($p > 0.05$ again). So adding the mortgage rate lag did not mess anything up. We do not really need the Newey-West correction here since there is no autocorrelation problem to fix, but we ran it anyway just to show how you would do it if a model did fail these tests.

4. Forecasting

To get a 16-quarter forecast we just keep plugging each new forecast back in as the next lag and repeat, since both models were fit on the differenced data.

```
sigma_ar  <- summary(ar_model)$sigma
sigma_ardl <- summary(ardl_model)$sigma
b_ar      <- coef(ar_model)
b_ardl    <- coef(ardl_model)

## AR(1) iterative forecast
ar_forecast_diff <- numeric(h)
last_dy <- as.numeric(tail(dy, 1))
for (i in 1:h) {
  next_val <- b_ar[1] + b_ar[2] * last_dy
  ar_forecast_diff[i] <- next_val
  last_dy <- next_val
}

## ARDL(1,1) iterative forecast -- we feed in the REAL observed mortgage-rate
## changes during the hold-out window, since the predictor is actually
## observed out-of-sample
dx_full <- diff(mort)
dx_holdout <- tail(dx_full, h)

ardl_forecast_diff <- numeric(h)
last_dy2 <- as.numeric(tail(dy, 1))
last_dx <- as.numeric(tail(dx, 1))
for (i in 1:h) {
  next_val <- b_ardl[1] + b_ardl[2] * last_dy2 + b_ardl[3] * last_dx
  ardl_forecast_diff[i] <- next_val
  last_dy2 <- next_val
  last_dx <- dx_holdout[i]
}

## Undo the differencing to get forecasts back in the original HOUST units
last_level <- as.numeric(tail(y_train, 1))
ar_forecast_level <- last_level + cumsum(ar_forecast_diff)
ardl_forecast_level <- last_level + cumsum(ardl_forecast_diff)

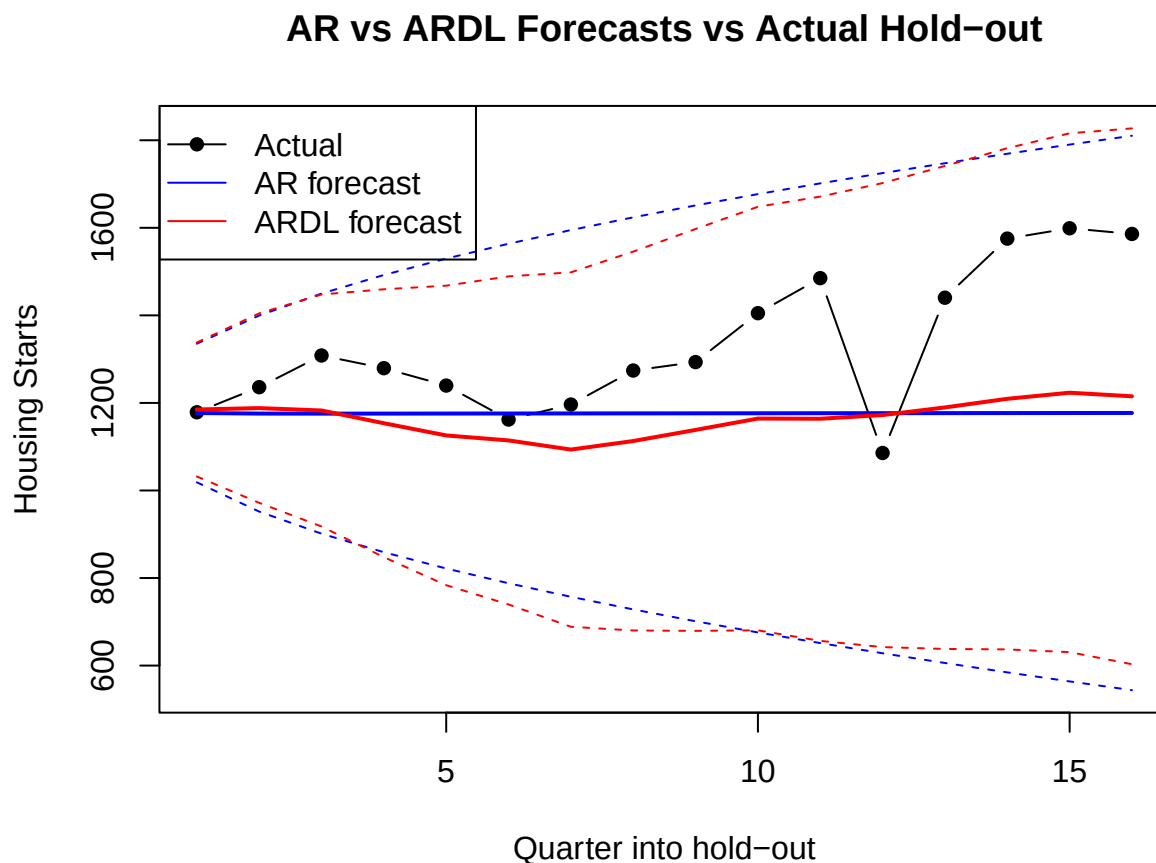
## Simple approximate 95% prediction interval that widens with the forecast
## horizon (uncertainty compounds as we chain one-step forecasts together)
```

```

steps <- 1:h
ar_upper <- ar_forecast_level + 1.96 * sigma_ar * sqrt(steps)
ar_lower <- ar_forecast_level - 1.96 * sigma_ar * sqrt(steps)
ardl_upper <- ardl_forecast_level + 1.96 * sigma_ardl * sqrt(steps)
ardl_lower <- ardl_forecast_level - 1.96 * sigma_ardl * sqrt(steps)

plot(1:h, as.numeric(y_test), type = "b", pch = 16,
     ylim = range(c(y_test, ar_upper, ar_lower, ardl_upper, ardl_lower)),
     xlab = "Quarter into hold-out", ylab = "Housing Starts",
     main = "AR vs ARDL Forecasts vs Actual Hold-out")
lines(1:h, ar_forecast_level, col = "blue", lwd = 2)
lines(1:h, ar_upper, col = "blue", lty = 2)
lines(1:h, ar_lower, col = "blue", lty = 2)
lines(1:h, ardl_forecast_level, col = "red", lwd = 2)
lines(1:h, ardl_upper, col = "red", lty = 2)
lines(1:h, ardl_lower, col = "red", lty = 2)
legend("topleft", legend = c("Actual", "AR forecast", "ARDL forecast"),
     col = c("black", "blue", "red"), lty = 1, pch = c(16, NA, NA))

```



For the first few hold-out quarters both forecasts stay pretty close to the actual values, but neither model catches the COVID dip in 2020 or the huge jump in housing starts in 2021. By the end of the window both forecast lines are way below where the actual data ends up. At least the actual values stay inside our 95% prediction interval the whole time, even though the interval gets pretty wide by then.

5. Out-of-Sample Accuracy

```
ar_rmse <- sqrt(mean((as.numeric(y_test) - ar_forecast_level)^2))
ar_mae <- mean(abs(as.numeric(y_test) - ar_forecast_level))
ardl_rmse <- sqrt(mean((as.numeric(y_test) - ardl_forecast_level)^2))
ardl_mae <- mean(abs(as.numeric(y_test) - ardl_forecast_level))

cat("AR RMSE:", ar_rmse, " MAE:", ar_mae, "\n")
```

```
## AR RMSE: 222.1067 MAE: 170.7943
```

```
cat("ARDL RMSE:", ardl_rmse, " MAE:", ardl_mae, "\n")
```

```
## ARDL RMSE: 217.0388 MAE: 181.1016
```

```
abs(as.numeric(y_test) - ar_forecast_level) ## per-step error, AR
```

```
## [1] 1.719728 60.304293 132.777203 103.703782 63.922882 13.865837
## [7] 20.343946 97.886775 117.096216 228.638979 308.515074 90.942165
## [13] 263.600596 398.476690 422.019451 408.895545
```

```
abs(as.numeric(y_test) - ardl_forecast_level) ## per-step error, ARDL
```

```
## [1] 6.337491 47.831934 125.604423 125.718163 113.942816 47.677275
## [7] 102.865202 161.079470 155.027691 240.689306 321.148683 86.576241
## [13] 250.673571 365.895137 375.686755 370.871613
```

ARDL gets the lower RMSE but AR actually gets the lower MAE, so honestly the two models are pretty close overall, ARDL is just a little better at handling the biggest misses. We are going with RMSE as our main metric here, so our preferred model is the **ARDL(1,1)**, since RMSE punishes big errors more and mortgage rates are clearly part of the story for housing starts. For both models the errors get bigger the further out we forecast, especially in the last few quarters that cover the COVID crash and the housing boom that came right after it. Neither model saw that coming since they were trained on a much calmer period from 1990-2017.

6. Limitations and Improvements

```
## Reverse-causality check: does the mortgage rate Granger-cause housing
## starts, or could housing activity itself feed back into rates?
grangertest(dy ~ dx, order = best_q)
```

```
## Granger causality test
```

```
##
```

```
## Model 1: dy ~ Lags(dy, 1:1) + Lags(dx, 1:1)
```

```
## Model 2: dy ~ Lags(dy, 1:1)
```

```
## Res.Df Df F Pr(>F)
```

```
## 1 105
```

```
## 2 106 -1 8.4952 0.004353 **
```

```
## ---
```

```
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```



```
grangertest(dx ~ dy, order = best_q)
```

```
## Granger causality test
##
## Model 1: dx ~ Lags(dx, 1:1) + Lags(dy, 1:1)
## Model 2: dx ~ Lags(dx, 1:1)
##   Res.Df Df       F Pr(>F)
## 1     105
## 2     106 -1 0.4529 0.5025
```

```
nobs(ardl_model)
```

```
## [1] 108
```

```
length(coef(ardl_model))
```

```
## [1] 3
```

The Granger test says mortgage rate changes Granger-cause housing starts ($p < 0.01$), but not the other way around, housing starts do not Granger-cause mortgage rate changes ($p > 0.05$). So it seems okay to treat mortgage rate as the variable that is driving things here, though this is just a 1-lag test and mortgage rates are set by the Fed, which reacts to the economy too, so reverse causality is still something to keep in mind. We still have about 108 usable quarters after differencing and lagging everything, and we are only estimating a couple of parameters, so we are not too worried about running out of degrees of freedom. That said, a longer sample or adding a second predictor like the unemployment rate would probably make this model stronger. Using a VAR instead would also let us actually test for feedback in both directions instead of just assuming mortgage rate is exogenous.

Part 2: Qualitative (Binary) Dependent Variable Models

Binary Dependent Variables

(a) Question and Data

For this part we want to see if we can predict whether a Titanic passenger survived, using their class, sex, age, number of siblings/spouses aboard, number of parents/children aboard, and how much they paid for their ticket.

Our data is the classic Titanic passenger list ($n = 891$), originally from the “Titanic: Machine Learning from Disaster” Kaggle competition and widely distributed as a standard teaching dataset (we pulled our copy from the `seaborn` sample-data repository). Each row is one passenger, with their class, sex, age, family size aboard, fare, and whether they survived the sinking. This is cross-sectional data (one snapshot, not tracked over time), which is why Part 2 uses a random hold-out split instead of the time-based split we used in Part 1.

```
titanic <- read.csv("titanic.csv")
titanic <- titanic[, c("survived", "pclass", "sex", "age", "sibsp", "parch", "fare")]
titanic$sex <- ifelse(titanic$sex == "male", 1, 0) ## 1 = male, 0 = female
```

(b) Missing data and hold-out split

```
colSums(is.na(titanic))
```

```
## survived  pclass      sex      age  sibsp  parch  fare
##          0         0         0     177     0     0     0
```

Age is missing for around 177 out of the 891 passengers. Since this data is not ordered by time like Part 1, using `na.locf()` here would not really make sense. Instead we fill in the missing ages with the median age from the training set. We compute that median after splitting the data so we do not accidentally leak test set info into training.

```
set.seed(104)
train_rows <- sample(1:nrow(titanic), size = 0.75 * nrow(titanic))
train <- titanic[train_rows, ]
test <- titanic[-train_rows, ]

median_age <- median(train$age, na.rm = TRUE)
train$age[is.na(train$age)] <- median_age
test$age[is.na(test$age)] <- median_age
median_age
```

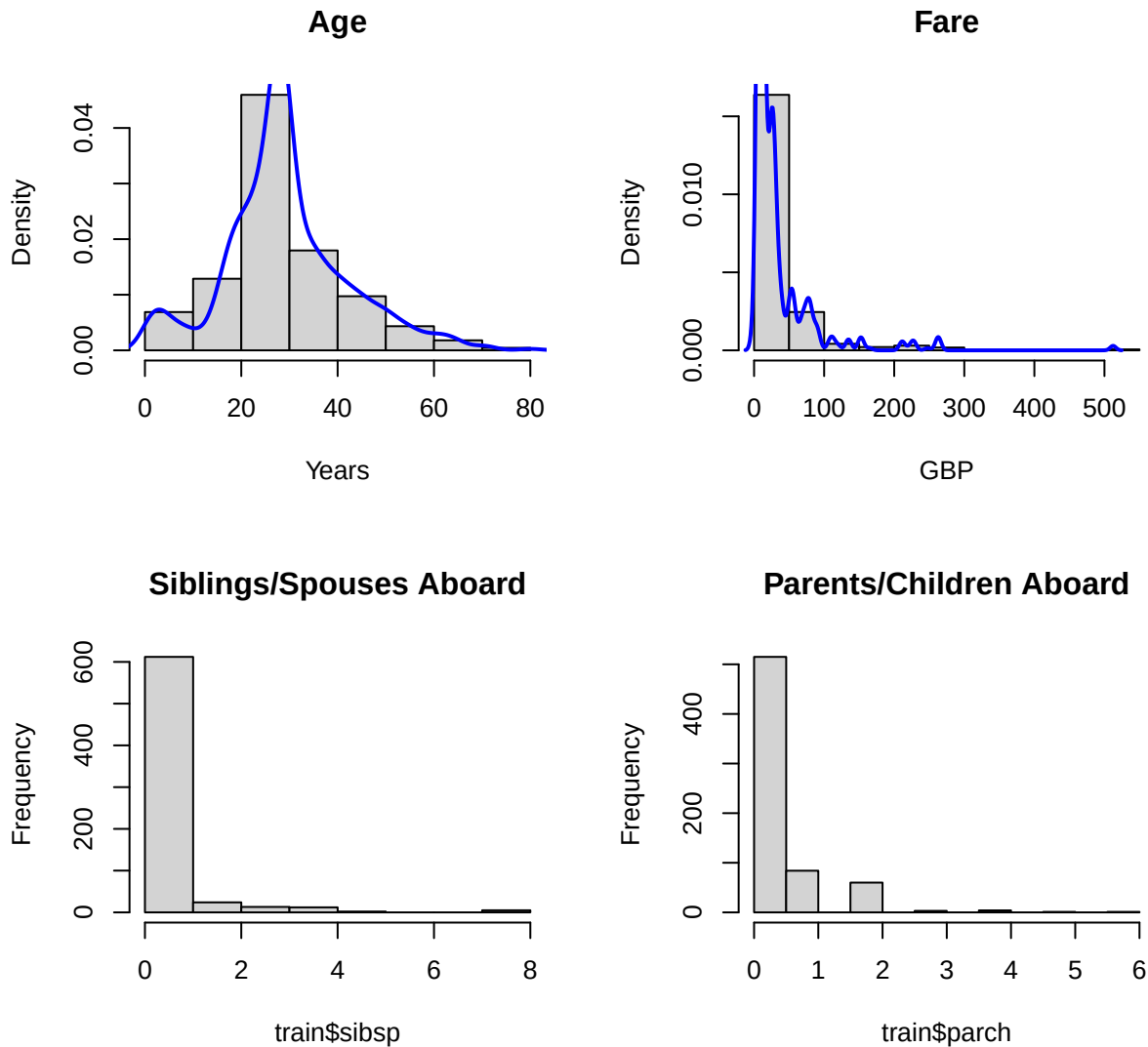
```
## [1] 28
```

(c) Descriptive analysis

```

par(mfrow = c(2, 2))
hist(train$age, freq = FALSE, main = "Age", xlab = "Years")
lines(density(train$age), col = "blue", lwd = 2)
hist(train$fare, freq = FALSE, main = "Fare", xlab = "GBP")
lines(density(train$fare), col = "blue", lwd = 2)
hist(train$sibsp, main = "Siblings/Spouses Aboard")
hist(train$parch, main = "Parents/Children Aboard")

```

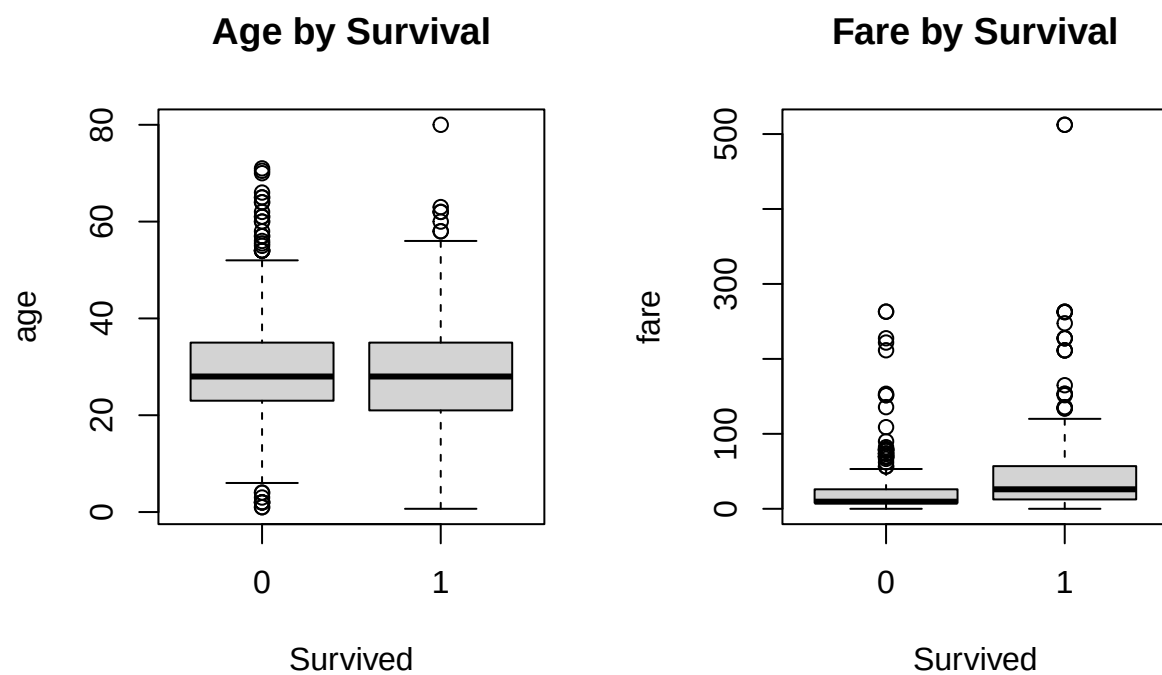


Age looks roughly bell-shaped and centered around the high-20s, though that is partly because of all the missing values we filled in with the median. Fare is really right-skewed, most tickets were cheap but there is a long tail of expensive first-class fares pulling it out. Most people were traveling with 0 siblings/spouses and 0 parents/children aboard, and the counts drop off fast as family size goes up.

```

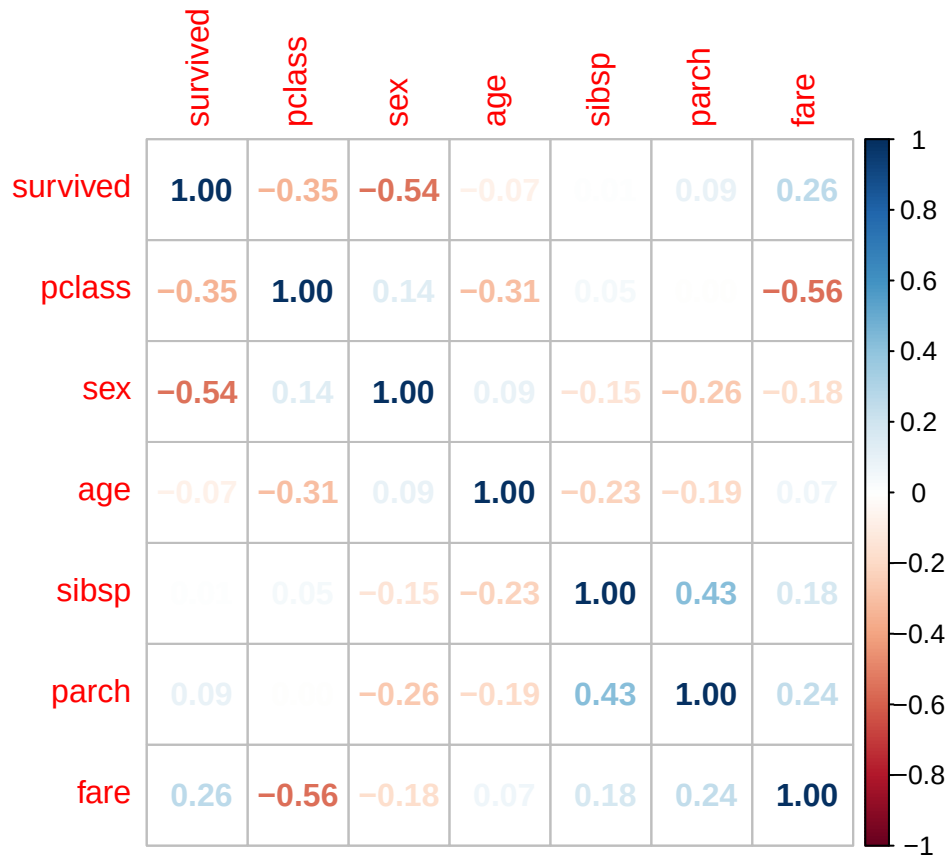
par(mfrow = c(1, 2))
boxplot(age ~ survived, data = train, main = "Age by Survival", xlab = "Survived")
boxplot(fare ~ survived, data = train, main = "Fare by Survival", xlab = "Survived")

```



Survivors look a little younger on average, and they clearly paid more for their tickets than the people who did not survive. Both groups have some high-fare outliers though.

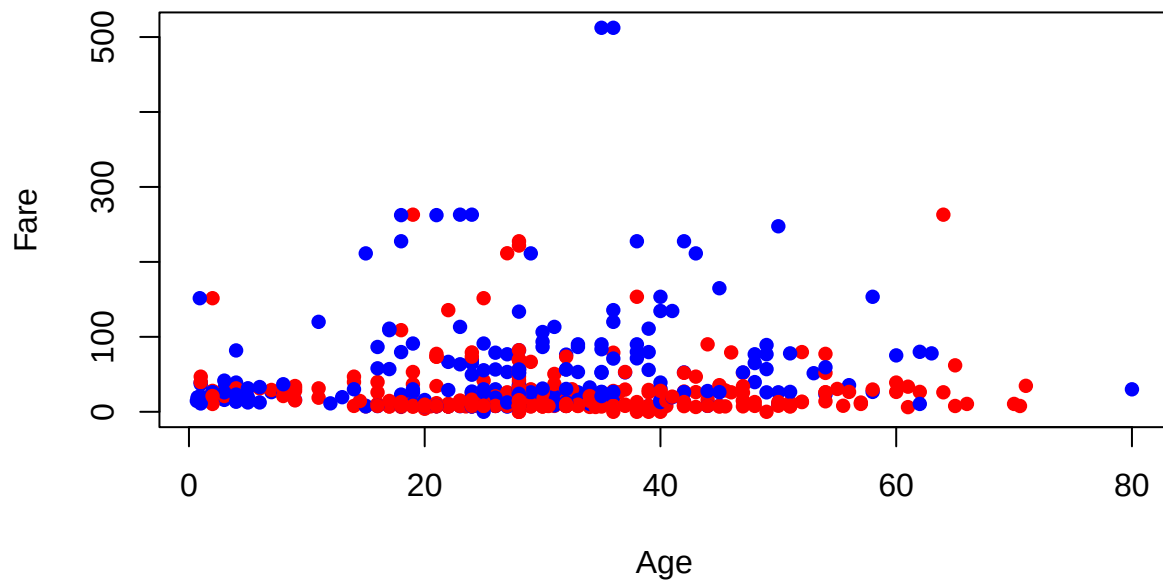
```
corrplot(cor(train), method = "number")
```



Sex and pclass have the strongest correlation with survival, around -0.5 and -0.3. This makes sense with the whole “women and children first” idea, and first class passengers getting priority over third class. Fare and pclass are also strongly correlated, about -0.55, which makes sense since fare basically tells you what class someone was in anyway.

```
plot(train$age, train$fare, col = ifelse(train$survived == 1, "blue", "red"),
     pch = 16, xlab = "Age", ylab = "Fare", main = "Age vs Fare, colored by Survival")
```

Age vs Fare, colored by Survival



The blue dots (survivors) tend to sit higher up on the fare axis no matter their age, while the red dots (non-survivors) are spread out more across the lower fares. This is basically the same fare and pclass story from the correlation plot showing up again here.

```
summary(train[, c("age", "fare", "sibsp", "parch")]) ## five-number summaries
```

```
##      age      fare      sibsp      parch
## Min.   : 0.67   Min.   : 0.000   Min.   :0.0000   Min.   :0.0000
## 1st Qu.:22.00   1st Qu.: 7.918   1st Qu.:0.0000   1st Qu.:0.0000
## Median :28.00   Median :14.427   Median :0.0000   Median :0.0000
## Mean   :29.24   Mean   :32.353   Mean   :0.4985   Mean   :0.3593
## 3rd Qu.:35.00   3rd Qu.:30.500   3rd Qu.:1.0000   3rd Qu.:0.0000
## Max.   :80.00   Max.   :512.329   Max.   :8.0000   Max.   :6.0000
```

```
prop.table(table(train$survived)) ## proportion who survived
```

```
##
##      0      1
## 0.6092814 0.3907186
```

```
prop.table(table(train$sex)) ## proportion male/female
```

```
##
##      0      1
## 0.3488024 0.6511976
```

```
prop.table(table(train$pclass))    ## class distribution
```

```
##
##      1      2      3
## 0.2365269 0.2140719 0.5494012
```

The five-number summaries back up what we saw in the histograms. Fare has a way longer right tail than age does, its max is around 35 times its median, while age's max is only around 3 times its median. Also most people have 0 for both sibsp and parch, since the median and 25th percentile are 0 for both. Looking at proportions, about 39% of people in our training set survived, about 65% were male, and just over half were in third class. So this was mostly a lower-class, mostly-male group of passengers.

(d) LPM, Probit, and Logit

```
lpm    <- lm(survived ~ pclass + sex + age + sibsp + parch + fare, data = train)
probit <- glm(survived ~ pclass + sex + age + sibsp + parch + fare, data = train,
              family = binomial(link = "probit"))
logit  <- glm(survived ~ pclass + sex + age + sibsp + parch + fare, data = train,
              family = binomial(link = "logit"))

summary(lpm)
```

```
##
## Call:
## lm(formula = survived ~ pclass + sex + age + sibsp + parch +
##     fare, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.08656 -0.18706 -0.09003  0.22569  0.97611
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.2965158  0.0818919  15.832  < 2e-16 ***
## pclass      -0.1720268  0.0231195  -7.441  3.12e-13 ***
## sex         -0.5129337  0.0327658 -15.655  < 2e-16 ***
## age        -0.0054901  0.0012752  -4.305  1.92e-05 ***
## sibsp       -0.0337068  0.0157670  -2.138   0.0329 *
## parch       -0.0289972  0.0225354  -1.287   0.1986
## fare         0.0004273  0.0003758   1.137   0.2559
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.3834 on 661 degrees of freedom
## Multiple R-squared:  0.3891, Adjusted R-squared:  0.3836
## F-statistic: 70.18 on 6 and 661 DF, p-value: < 2.2e-16
```

```
summary(probit)
```

```
##
```

```
## Call:
## glm(formula = survived ~ pclass + sex + age + sibsp + parch +
##      fare, family = binomial(link = "probit"), data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  2.791020   0.337541   8.269 < 2e-16 ***
## pclass      -0.609086   0.089174  -6.830 8.47e-12 ***
## sex         -1.618014   0.128204 -12.621 < 2e-16 ***
## age         -0.020599   0.005139  -4.009 6.11e-05 ***
## sibsp       -0.147424   0.069214  -2.130  0.0332 *
## parch       -0.115859   0.084762  -1.367  0.1717
## fare         0.001683   0.001466   1.148  0.2508
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 893.88  on 667  degrees of freedom
## Residual deviance: 602.36  on 661  degrees of freedom
## AIC: 616.36
##
## Number of Fisher Scoring iterations: 5
```

```
summary(logit)
```

```
##
## Call:
## glm(formula = survived ~ pclass + sex + age + sibsp + parch +
##      fare, family = binomial(link = "logit"), data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  4.855310   0.603536   8.045 8.64e-16 ***
## pclass      -1.079975   0.157115  -6.874 6.25e-12 ***
## sex         -2.730825   0.229118 -11.919 < 2e-16 ***
## age         -0.036540   0.009081  -4.024 5.73e-05 ***
## sibsp       -0.256388   0.122032  -2.101  0.0356 *
## parch       -0.172139   0.143823  -1.197  0.2314
## fare         0.002597   0.002508   1.035  0.3005
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 893.88  on 667  degrees of freedom
## Residual deviance: 601.39  on 661  degrees of freedom
## AIC: 615.39
##
## Number of Fisher Scoring iterations: 5
```

```
pR2(probit)  ## McFadden's R-squared
```

```
## fitting null model for pseudo-r2
```



```
##          llh          llhNull          G2          McFadden          r2ML          r2CU
## -301.1824888 -446.9377071 291.5104368 0.3261198 0.3536363 0.4793989
```

```
pR2(logit)
```

```
## fitting null model for pseudo-r2
```

```
##          llh          llhNull          G2          McFadden          r2ML          r2CU
## -300.6942526 -446.9377071 292.4869091 0.3272122 0.3545804 0.4806788
```

In all three models, pclass, sex, age, and sibsp all come out significant ($p < 0.05$), and the signs agree across models. Being in a lower class (a higher pclass number), being male, being older, and having more siblings/spouses aboard are all linked to a LOWER chance of survival. parch and fare do not come out significant in any of the three models, probably because fare is already telling us basically the same thing as pclass.

For magnitude, the LPM coefficients can be read directly as “percentage point change in survival probability,” so e.g. being male is associated with roughly a 50 percentage point drop in survival probability, holding the other variables fixed (check your own `summary(lpm)` output for the exact number). The probit and logit coefficients are not on that same probability scale (they are in z-score and log-odds units), so we cannot compare their raw numbers to the LPM’s directly, only the sign, significance, and relative ranking of predictors line up across all three. All three models agree that sex has by far the largest effect on survival, followed by pclass, then age and sibsp, with parch and fare mattering the least. The main practical issue with LPM is that it can predict probabilities below 0 or above 1, which does not really make sense for a probability, while probit and logit avoid that problem by design.

(e) Hold-out evaluation

We now take all three fitted models, generate predicted probabilities on the hold-out set, and classify a passenger as “survived” if the predicted probability is at least 0.5.

LPM

```
lpm_pred_prob <- predict(lpm, newdata = test)
lpm_pred_class <- ifelse(lpm_pred_prob >= 0.5, 1, 0)
lpm_cm <- confusionMatrix(factor(lpm_pred_class), factor(test$survived), positive = "1")
lpm_cm
```

```
## Confusion Matrix and Statistics
```

```
##
##          Reference
## Prediction  0    1
##          0 123  25
##          1  19  56
##
##          Accuracy : 0.8027
##          95% CI : (0.7443, 0.8528)
##    No Information Rate : 0.6368
##    P-Value [Acc > NIR] : 5.285e-08
##
##          Kappa : 0.5666
##
```

```
## McNemar's Test P-Value : 0.451
##
##           Sensitivity : 0.6914
##           Specificity : 0.8662
##           Pos Pred Value : 0.7467
##           Neg Pred Value : 0.8311
##           Prevalence : 0.3632
##           Detection Rate : 0.2511
##           Detection Prevalence : 0.3363
##           Balanced Accuracy : 0.7788
##
##           'Positive' Class : 1
##
```

```
lpm_roc <- roc(test$survived, lpm_pred_prob)
auc(lpm_roc)
```

```
## Area under the curve: 0.864
```

Probit

```
probit_pred_prob <- predict(probit, newdata = test, type = "response")
probit_pred_class <- ifelse(probit_pred_prob >= 0.5, 1, 0)
probit_cm <- confusionMatrix(factor(probit_pred_class), factor(test$survived), positive = "1")
probit_cm
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0 121  23
##           1  21  58
##
##           Accuracy : 0.8027
##           95% CI : (0.7443, 0.8528)
##           No Information Rate : 0.6368
##           P-Value [Acc > NIR] : 5.285e-08
##
##           Kappa : 0.5712
##
## McNemar's Test P-Value : 0.8802
##
##           Sensitivity : 0.7160
##           Specificity : 0.8521
##           Pos Pred Value : 0.7342
##           Neg Pred Value : 0.8403
##           Prevalence : 0.3632
##           Detection Rate : 0.2601
##           Detection Prevalence : 0.3543
##           Balanced Accuracy : 0.7841
##
##           'Positive' Class : 1
##
```

```
probit_roc <- roc(test$survived, probit_pred_prob)
auc(probit_roc)
```

Area under the curve: 0.8662

Logit

```
logit_pred_prob <- predict(logit, newdata = test, type = "response")
logit_pred_class <- ifelse(logit_pred_prob >= 0.5, 1, 0)
logit_cm <- confusionMatrix(factor(logit_pred_class), factor(test$survived), positive = "1")
logit_cm
```

Confusion Matrix and Statistics

```
##
##              Reference
## Prediction    0    1
##              0 120  23
##              1  22  58
##
##              Accuracy : 0.7982
##              95% CI : (0.7395, 0.8488)
##      No Information Rate : 0.6368
##      P-Value [Acc > NIR] : 1.224e-07
##
##              Kappa : 0.5626
##
##  McNemar's Test P-Value : 1
##
##              Sensitivity : 0.7160
##              Specificity : 0.8451
##              Pos Pred Value : 0.7250
##              Neg Pred Value : 0.8392
##              Prevalence : 0.3632
##              Detection Rate : 0.2601
##      Detection Prevalence : 0.3587
##              Balanced Accuracy : 0.7806
##
##              'Positive' Class : 1
##
```

```
logit_roc <- roc(test$survived, logit_pred_prob)
auc(logit_roc)
```

Area under the curve: 0.8656

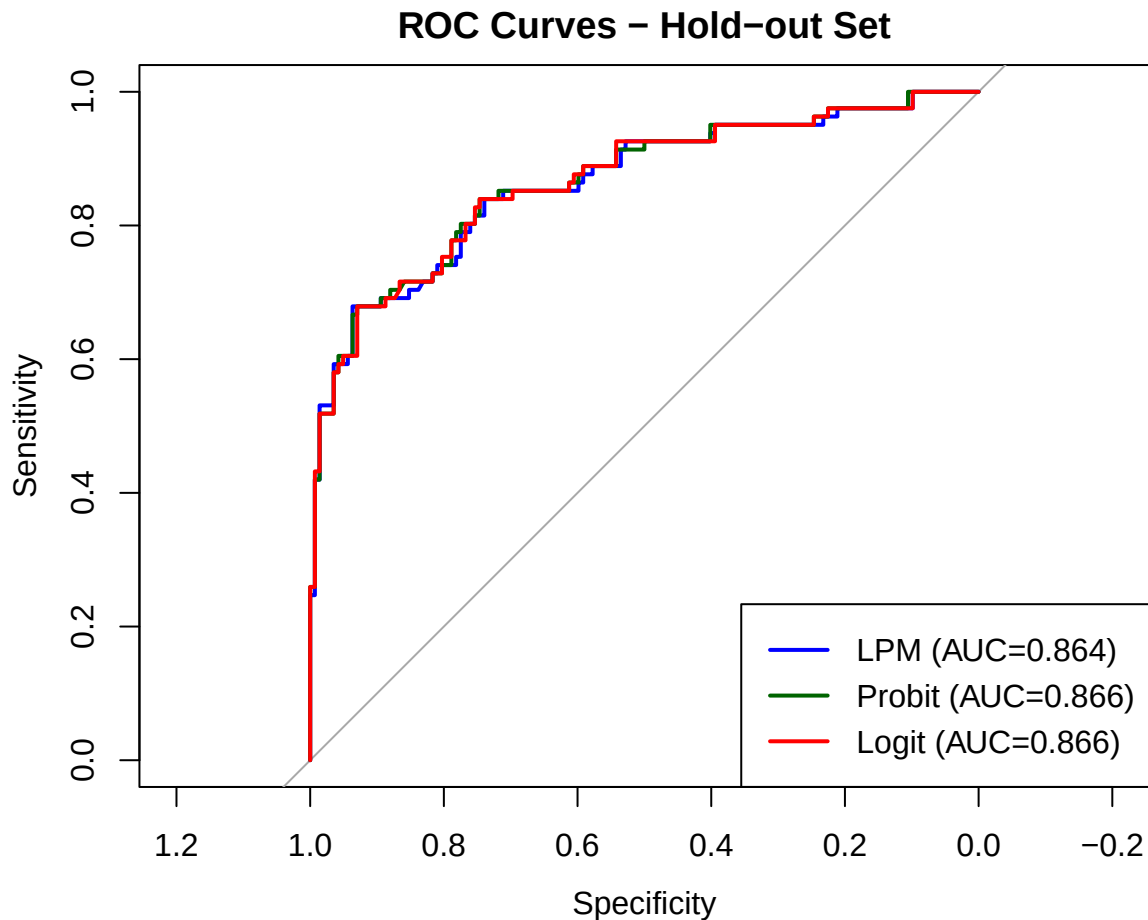
ROC curves, all three models overlaid

```
plot(lpm_roc, col = "blue", main = "ROC Curves - Hold-out Set")
plot(probit_roc, col = "darkgreen", add = TRUE)
plot(logit_roc, col = "red", add = TRUE)
legend("bottomright",
```

```

legend = c(paste0("LPM (AUC=", round(auc(lpm_roc), 3), ")"),
           paste0("Probit (AUC=", round(auc(probit_roc), 3), ")"),
           paste0("Logit (AUC=", round(auc(logit_roc), 3), ")")),
col = c("blue", "darkgreen", "red"), lwd = 2)

```



The results of the three models are similar on the hold-out set. Accuracy ranges from the high .70s to low .80s. AUC ranges from the mid-to-high .80s. Sensitivity is much smaller than specificity for each model, meaning they are all better at identifying patients that did not survive than those that did. This makes sense given the roughly 62/38 split we have for the classes in the training set. For our purposes, we will choose **Logit** as our model of choice because its predicted probabilities always range from 0 to 1 (unlike LPM), its coefficients can easily be transformed to odds ratios by taking `exp(coef(logit))`, and its accuracy and AUC are essentially tied with the other two models.